Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота № 2

з дисципліни «Життєвий цикл розробки програмного забезпечення»

Тема: «Проектування архітектури та деталізоване проектування»

Виконали:
студентки групи ІП-44
Бідюк Марія
Хмелюк Діана

Перевірив:
Альбрехт Йосип
Омелянович

Київ 2026

ЗМІСТ

1. ВСТУП ТА МЕТА ПРОЄКТУВАННЯ..... 3

2. ВИБІР ТА ОБҐРУНТУВАННЯ АРХІТЕКТУРНОГО ПІДХОДУ..... 3

3. ВИСОКОРІВНЕВА АРХІТЕКТУРА СИСТЕМИ (COMPONENT DIAGRAM)..... 4

4. ДЕТАЛІЗОВАНЕ ПРОЄКТУВАННЯ КОМПОНЕНТІВ (UML-ДІАГРАМИ)..... 5

4.1. Статична структура: Діаграма класів (Class Diagram)..... 7

4.2. Динамічна поведінка: Діаграма послідовностей (Sequence Diagram). 9

4.3. Життєвий цикл сутності: Діаграма станів (State Machine Diagram) 11

5. ОПИС ВЗАЄМОДІЇ КОМПОНЕНТІВ ТА ЗАХИСТ НЕФУНКЦІОНАЛЬНИХ ВИМОГ..... 13

6. ВИСНОВКИ..... 13

1. ВСТУП ТА МЕТА ПРОЄКТУВАННЯ

Метою даної лабораторної роботи є перетворення вимог, сформованих у специфікації SRS (Лабораторна робота №1), у повноцінне архітектурне рішення програмного забезпечення. Проєкт розробляється як веб-застосунок для знайомств, ключовими особливостями якого є **розподіл даних анкети на Відкриту та Закриту інформацію** (із доступом за взаємною згодою) та **автоматичний розрахунок сумісності за нумерологічною матрицею долі**.

Для забезпечення надійності, масштабованості та безпеки системи розроблено високорівневу макроархітектуру та деталізовані UML-моделі статичної й динамічної поведінки компонентів.

2. ВИБІР ТА ОБҐРУНТУВАННЯ АРХІТЕКТУРНОГО ПІДХОДУ

На основі специфікації вимог для реалізації системи обрано **архітектуру модульного моноліту (Monolithic Architecture)** на базі фреймворку **Spring Boot.**

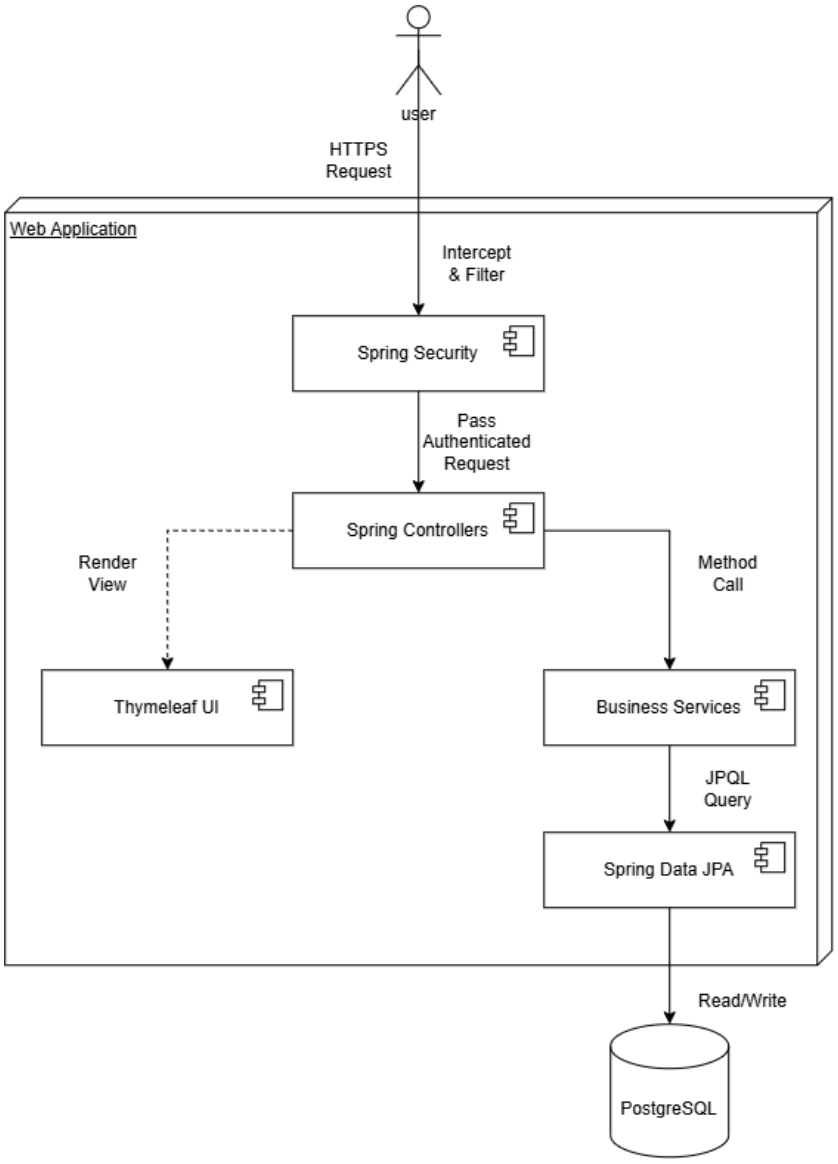
Обґрунтування вибору:

1. **Забезпечення продуктивності:** відсутність мережевих затримок між окремими мікросервісами дозволяє виконувати розрахунки та обробку запитів у межах **до 2 секунд** для завантаження сторінок та **до 1 секунди** для обчислення матриці долі.
2. **Єдиний контекст безпеки:** використання **Spring Security** у межах монолітного застосунку дозволяє прозоро перехоплювати вхідні запити (SecurityFilterChain), керувати сесіями користувачів та гарантувати, що чутливі закриті дані фізично не потрапляють на клієнтську сторону до підтвердження запрошення.
3. **Простота розгортання та підтримки:** на початковому етапі розвитку продукту (MVP) монолітна структура суттєво спрощує

налаштування бази даних **PostgreSQL** та знижує накладні витрати на інфраструктурний моніторинг.

3. ВИСОКОРІВНЕВА АРХІТЕКТУРА СИСТЕМИ (COMPONENT DIAGRAM)

Високорівнева макроархітектура зображує розподіл системи на шари та схему проходження вхідного HTTPS-запиту від користувача.



Опис компонентів та шарів:

- **Security Layer (Spring Security):** перехоплює HTTPS-запити від користувача (Intercept & Filter), здійснює перевірку автентифікації та передає перевірений запит на рівень контролерів.

- **Presentation Layer (Spring Controllers & Thymeleaf UI):** отримує валідовані запити, викликає методи бізнес-сервісів та рендерить HTML-шаблони для користувача.
- **Business Layer (Business Services):** реалізує бізнес-логіку перевірки прав доступу, обміну запрошеннями та розрахунку матриці долі.
- **Persistence Layer (Spring Data JPA):** здійснює формування JPQL-запитів та обмін даними із реляційною СУБД PostgreSQL.

4. ДЕТАЛІЗОВАНЕ ПРОЄКТУВАННЯ КОМПОНЕНТІВ (UML-ДІАГРАМИ)

Відповідно до вимог методичних вказівок та стандартів проєктування складних програмних систем (ISO/IEC/IEEE 42010), деталізація високорівневої архітектури вимагає опису системи у трьох фундаментальних вимірах: **статичному (структура даних), динамічному міжшаровому (поведінка компонентів у часі)** та **динамічному об'єктному (життєвий цикл сутності)**.

Для даного проєкту обрано комплект із трьох UML-діаграм – **Діаграму класів (Class Diagram), Діаграму послідовностей (Sequence Diagram) та Діаграму станів (State Machine Diagram)** – з наступних архітектурних міркувань:

1. Діаграма класів (Class Diagram) – обґрунтування статичного аспекту:

- **критичність для проєкту:** ключовою особливістю системи є розмежування атрибутів анкети на відкриті (openData) та закриті (closedData), а також складні зв'язки між користувачами, анкетними інтересами та результатами нумерологічних обчислень;
- **чому це необхідно:** саме діаграма класів дозволяє зафіксувати контракти даних, типи атрибутів, модифікатори доступу (private/public) та JPA-відносини (@OneToOne, @ManyToMany, @ManyToOne). Вона є

прямим містком між вимогами приватності SRS та практичною побудовою реляційної бази даних PostgreSQL і Java-сутностей.

2. Діаграма послідовностей (Sequence Diagram) – обґрунтування динамічного аспекту:

- **критичність для проєкту:** головний бізнес-сценарій системи (UC-1: обмін запрошеннями та розрахунок сумісності) є скрізним і залучає одночасно 7 компонентів: двох акторів, UI, Spring MVC Контролери, бізнес-сервіси (InvitationService, DestinyMatrixService), JPA-репозиторій та СУБД PostgreSQL;
- **чому це необхідно:** діаграма послідовностей чітко документує часовий порядок виклику методів, міжшаровий обмін DTO-об'єктами та атомарність транзакцій. Вона демонструє, як саме Spring Security та бізнес-логіка блокують доступ до закритих даних до моменту настання події підтвердження запрошення.

3. Діаграма станів (State Machine Diagram) – обґрунтування динамічного об'єктного аспекту:

- **критичність для проєкту:** сутність запрошення (Invitation) є центральним елементом механізму розмежування приватності, яка змінює свій стан у відповідь на дії обох користувачів (надсилання, прийняття, відхилення або скасування);
- **чому це необхідно:** діаграма станів наочно документує життєвий цикл об'єкта Invitation через Enum InvitationStatus (PENDING → ACCEPTED / REJECTED / CANCELLED), фіксуючи бізнес-правила приватності та тригери переходів для всіх можливих гілок бізнес-логіки.

4. Синергія та повнота покриття комплекту з 3-х діаграм:

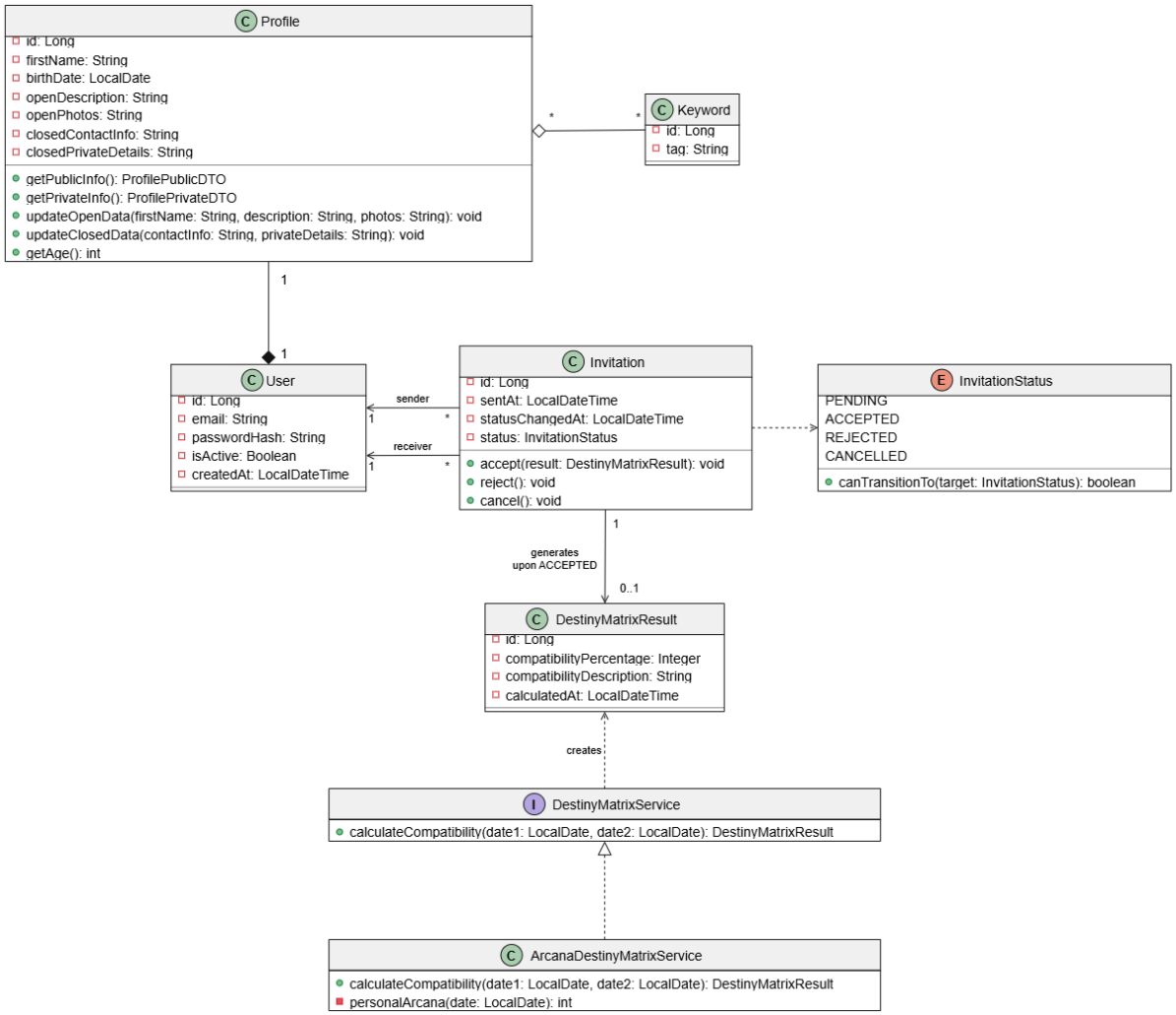
Поєднання Class Diagram + Sequence Diagram + State Machine Diagram надає 100% необхідної інформації для розробників та тестувальників:

- Class Diagram визначає, що і як зберігати;

- Sequence Diagram – хто, коли і в якому порядку обробляє запити;
- State Machine Diagram – як поводиться головна сутність на кожному етапі свого життя.

4.1. Статична структура: Діаграма класів (Class Diagram)

Діаграма класів відображає об'єктно-орієнтовану модель системи, її атрибути, типи даних, методи та зв'язки між сутностями.



Ключові елементи моделі:

1. **User (сутність облікового запису):** містить ідентифікатор id, email, passwordHash, прапорець isActive та дату створення createdAt. Сутність зберігає лише дані облікового запису: реєстрацію виконує сервіс UserService, а вхід у систему – Spring Security.

2. **Profile (сутність анкети):** містить ім'я firstName і дату народження birthDate (використовується для розрахунку матриці долі, іншим користувачам показується лише вік – метод getAge()), а також фізично розділяє дані на відкриті (openDescription, openPhotos) та закриті (closedContactInfo, closedPrivateDetails). Метод getPublicInfo() повертає лише відкриту частину (ProfilePublicDTO), getPrivateInfo() – закриту (ProfilePrivateDTO); право доступу до закритої частини перевіряє бізнес-сервіс до виклику цього методу.

3. **Keyword (теги/інтереси):** довідник інтересів користувача, пов'язаний із Profile відношенням «багато-до-багатьох».

4. **Invitation (запрошення):** зберігає дату відправлення sentAt, дату зміни стану statusChangedAt та поточний стан із перелічення InvitationStatus (PENDING, ACCEPTED, REJECTED, CANCELLED). Методи accept(result), reject() і cancel() змінюють стан лише з PENDING, що перевіряє метод InvitationStatus.canTransitionTo().

5. **DestinyMatrixService (інтерфейс сервісу обчислення сумісності):** оголошує метод calculateCompatibility(date1: LocalDate, date2: LocalDate): DestinyMatrixResult. Його реалізує клас ArcanaDestinyMatrixService, який обчислює нумерологічну матрицю долі. Завдяки інтерфейсу алгоритм розрахунку можна замінити без змін у бізнес-логіці запрошень (принципи Open/Closed та Dependency Inversion).

6. **DestinyMatrixResult (сутність результату сумісності):** збережена сутність даних (POJO), яка містить відсоток сумісності compatibilityPercentage, опис compatibilityDescription та дату розрахунку calculatedAt.

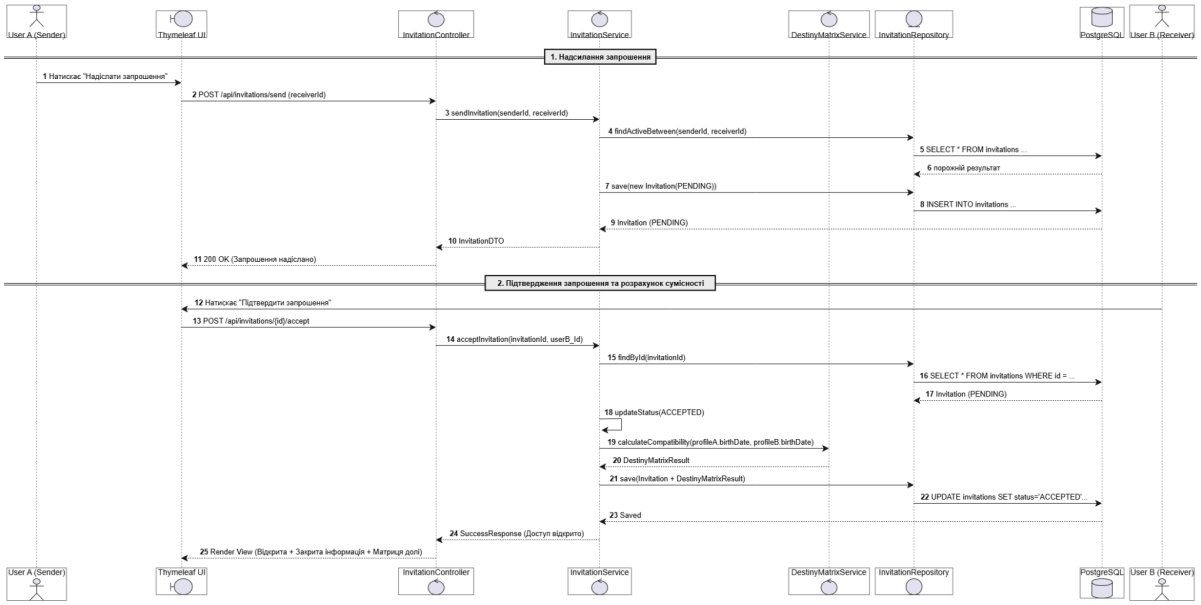
Типи зв'язків та їхнє обґрунтування:

- **User – Profile (композиція):** заповнений чорний ромб вказує, що анкета належить єдиному користувачу і не існує без нього.

- **Profile – Keyword (агрегація):** порожній ромб вказує, що теги інтересів є окремим довідником і зберігаються у системі навіть при видаленні анкети.
- **User – Invitation (асоціація):** дві спрямовані асоціації з кратністю 1 до *, що розділяють ролі відправника та отримувача запрошення.
- **Invitation – DestinyMatrixResult (асоціація):** кратність 1 до 0...1 показує, що розрахунок матриці долі створюється лише при переході запрошення у статус ACCEPTED.
- **Invitation – InvitationStatus (залежність):** пунктирна стрілка показує використання класів-перелічень станів.
- **DestinyMatrixService – DestinyMatrixResult (залежність):** пунктирна стрілка з підписом creates від класу DestinyMatrixService напряму до DestinyMatrixResult, яка показує, що сервіс формує та обчислює об'єкт результату.
- **ArcanaDestinyMatrixService – DestinyMatrixService (реалізація):** пунктирна стрілка з порожнім трикутником показує, що клас реалізує інтерфейс сервісу розрахунку сумісності.

4.2. Динамічна поведінка: Діаграма послідовностей (Sequence Diagram)

Діаграма послідовностей відображає виконання у часі головного бізнес-сценарію UC-1: «Обмін запрошеннями, розкриття закритих даних та розрахунок сумісності за матрицею долі».



Покроковий перебіг сценарію:

- **Блок 1: Надсилання запрошення (кроки 1–11)**

1. Користувач A у формі Thymeleaf UI натискає кнопку «Надіслати запрошення».
2. На InvitationController надходить запит POST /api/invitations/send.
3. Контролер викликає метод sendInvitation() у бізнес-сервісі InvitationService.
4. InvitationService звертається до InvitationRepository через метод findActiveBetween().
5. InvitationRepository виконує SQL-запит SELECT у СУБД PostgreSQL.
6. PostgreSQL повертає порожній результат (підтверджуючи відсутність дублюючих активних запрошень).
7. InvitationService створює новий об'єкт Invitation зі статусом PENDING.
8. InvitationRepository виконує запит INSERT у PostgreSQL.
9. PostgreSQL підтверджує успішне збереження.
10. InvitationService повертає InvitationDTO до контролера.

11. InvitationController повертає на UI статус 200 OK (Запрошення надіслано).

- **Блок 2: Підтвердження та розрахунок сумісності (кроки 12–25)**

12. Користувач Б у своєму інтерфейсі Thymeleaf UI натискає «Підтвердити запрошення».

13. На InvitationController надходить запит POST /api/invitations/{id}/accept.

14. Контролер викликає метод acceptInvitation() у InvitationService.

15. InvitationService робить запит findById() до InvitationRepository.

16. InvitationRepository завантажує актуальний стан з PostgreSQL.

17. PostgreSQL повертає об'єкт запрошення зі статусом PENDING.

18. InvitationService змінює внутрішній статус об'єкта на ACCEPTED.

19. InvitationService викликає calculateCompatibility() у DestinyMatrixService, передаючи дати народження обох користувачів.

20. DestinyMatrixService обчислює сумісність та повертає об'єкт DestinyMatrixResult.

21. InvitationService зв'язує результат із запрошенням та викликає save() у InvitationRepository.

22. InvitationRepository оновлює запис (UPDATE) у PostgreSQL.

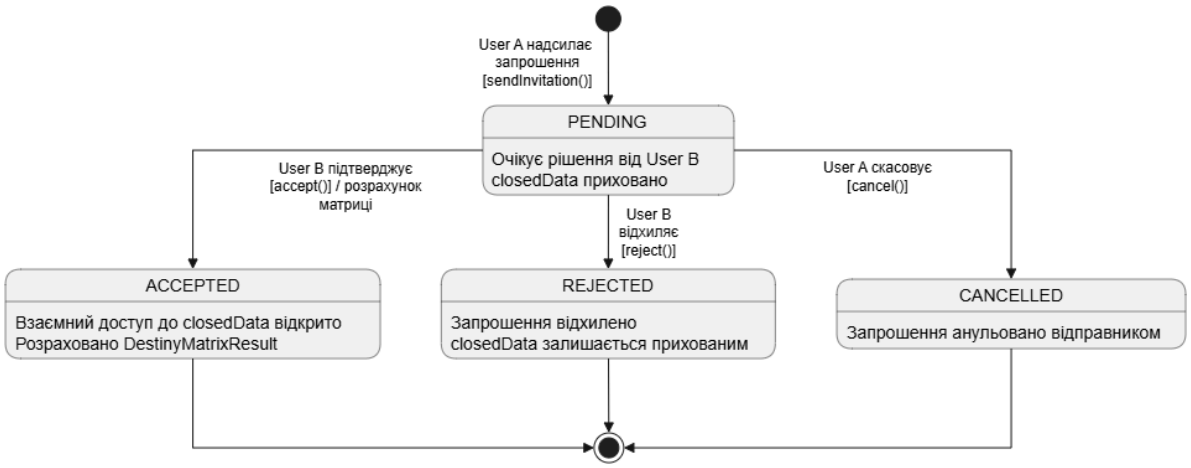
23. PostgreSQL підтверджує збереження.

24. InvitationService повертає SuccessResponse до контролера.

25. InvitationController рендерить подання Thymeleaf UI, відкриваючи користувачам взаємний доступ до Відкритої + Закритої інформації та розрахованих показників Матриці долі.

4.3. Життєвий цикл сутності: Діаграма станів (State Machine Diagram)

Для деталізації правил зміни станів ключового об'єкта предметної області – запрошення (Invitation) – побудовано діаграму станів. Вона описує переходи між значеннями перелічення InvitationStatus під впливом дій користувачів та бізнес-правил системи приватності.



Опис станів та переходів:

1. **Початковий стан → PENDING (очікування):**

- **тригер:** користувач А надсилає запит через метод sendInvitation();
- **особливість:** запрошення отримує стан PENDING. На цьому етапі закриті дані обох користувачів (closedContactInfo, closedPrivateDetails) залишаються повністю прихованими.

2. **Перехід PENDING → ACCEPTED (прийнято):**

- **тригер:** користувач Б натискає «Підтвердити» (метод accept());

- **побічний ефект:** автоматично запускається DestinyMatrixService.calculateCompatibility(), зберігається DestinyMatrixResult, і система відкриває взаємний доступ до закритих даних анкет.

3. **Перехід PENDING → REJECTED (відхилено):**

- **тригер:** користувач Б натискає «Відхилити» (метод reject());
- **особливість:** доступ до закритих даних блокується, розрахунок сумісності не виконується.

4. **Перехід PENDING → CANCELLED (скасовано):**

- **тригер:** користувач А скасовує власне запрошення до отримання відповіді (метод cancel());
- **особливість:** запрошення переходить у термінальний стан анулювання.

5. ОПИС ВЗАЄМОДІЇ КОМПОНЕНТІВ ТА ЗАХИСТ НЕФУНКЦІОНАЛЬНИХ ВИМОГ

Розроблена архітектура гарантує виконання всіх нефункціональних вимог специфікації SRS:

1. **Безпека та приватність:**

- закрита інформація (closedContactInfo, closedPrivateDetails) фізично відокремлена від відкритих DTO і передається з бекенду лише після перевірки статусу запрошення ACCEPTED у InvitationService;
- паролі користувачів хешуються алгоритмом BCrypt перед збереженням у User.

2. **Продуктивність та швидкодія:**

- розрахунок матриці долі винесено в окремий легковаговий сервіс DestinyMatrixService, що гарантує час обчислення до 1 секунди;
- використання JPA-індексів у PostgreSQL за ключовими словами забезпечує завантаження пошукових вибірок до 2 секунд.

3. **Надійність:**

- транзакційність у InvitationService (@Transactional) гарантує, що зміна статусу запрошення та створення запису матриці долі виконуються атомарно.

6. ВИСНОВКИ

У ході виконання Лабораторної роботи №2 було розроблено повноцінний документ «Архітектура програмного забезпечення» та деталізовані UML-моделі для веб-застосунку «Сайт знайомств з розрахунком сумісності за матрицею долі».

Основні результати роботи:

1. **Обґрунтування макроархітектури:** обрано та обґрунтовано підхід модульного моноліту на базі Spring Boot, Thymeleaf та PostgreSQL. Дане рішення є оптимальним для розробки командою з двох осіб, оскільки забезпечує чітку ізоляцію доменних модулів, зводить до мінімуму конфлікти у версіонуванні коду (Git), гарантує високу швидкодію (завантаження сторінок до 2 секунд, розрахунок сумісності до 1 секунди) та простоту розгортання єдиного артефакту.

2. **Високорівневе проєктування (Component Diagram):** побудовано діаграму компонентів, яка демонструє багатошарову структуру системи та трасування вхідного HTTPS-запиту крізь периметр безпеки (Spring Security), рівень представлення (Controllers / Thymeleaf UI), сервіси бізнес-логіки (Services) та шар абстракції даних (Spring Data JPA / PostgreSQL).

3. **Деталізація статичної структури (Class Diagram v2.0):** створено оновлену модель класів, яка реалізує ключову вимогу приватності (розділення анкетних даних на openData та closedData). Для дотримання принципу єдиної відповідальності (Single Responsibility) обчислювальний функціонал винесено в окремий інтерфейс DestinyMatrixService з реалізацією ArcanaDestinyMatrixService, яка формує

сутність DestinyMatrixResult та зафіксовано коректні UML-зв'язки (композиція, агрегація, спрямовані асоціації та залежності).

4. **Деталізація міжшарової динаміки (Sequence Diagram):** побудовано діаграму послідовностей для головного бізнес-сценарію UC-1 (25 послідовних кроків), яка відображає часовий порядок виклику методів, обмін DTO-об'єктами та транзакційну взаємодію 7 компонентів системи під час надсилання, підтвердження запрошення та автоматичного обчислення сумісності.

5. **Деталізація життєвого циклу сутності (State Machine Diagram):** додано діаграму станів для центрального об'єкта приватності – Invitation. Вона описує всі можливі переходи між значеннями Enum InvitationStatus (PENDING → ACCEPTED / REJECTED / CANCELLED), фіксуючи умови взаємного розкриття закритих даних та розгалуження бізнес-логіки.

Побудований комплекс UML-моделей гарантує дотримання нефункціональних вимог безпеки, надійності та продуктивності і слугує готовим технічним завданням для скрам-планування беклогу, розробки кодової бази, інтеграції компонентів та підготовки тест-кейсів.